

Práctica N° 1

Informática Gráfica y Visualización

Visualización 3D

Contenidos:

- Primitivas básicas.
- Transformaciones geométricas, composición de matrices, pila de matrices.
- La cámara sintética, proyección, viewport.

Objetivos:

- Situar distintas primitivas sobre una escena y aplicar transformaciones geométricas.
- Situar una cámara sobre la escena e implementar distintos movimientos de la cámara y cambio de parámetros.

Número de sesiones de laboratorio para terminar la práctica: 4 sesiones (8 horas). En la última sesión se realizará un examen práctico.

Puntuación: 3.5 sobre 10, luego ponderado al 50% (calificación prácticas).

Explicación:

A lo largo de dos de las sesiones de prácticas, al principio de las sesiones según se avance en las clases de teoría, el profesorado explicará los conceptos fundamentales para el desarrollo de la práctica, basándose en el código resuelto de los enunciados desarrollados al final de este documento (Ejemplos explicativos).

En base a este código, implementado mediante clases en C++, que utilizan la librería OpenGL en el entorno de desarrollo instalado en los laboratorios docentes, es necesario desarrollar un programa que resuelva lo indicado en esta práctica sobre Visualización 3D.

Deben generarse 3 objetos distintos, conformado por varias partes e implementados con primitivas básicas como cubos, cilindros, conos, esferas, etc. Estos objetos deben visualizarse juntos de manera simultánea como un todo (en esta práctica no se tratan aún objetos articulados).

Los objetos generados deben poder seleccionarse utilizando el teclado (asociando un número a cada uno de los objetos: “1”, “2”, “3”) y una vez seleccionado uno de los objetos permitir aplicarle transformaciones (traslación, rotación y escalado), utilizando el teclado, de manera que se vayan acumulando las transformaciones de modo independiente para Traslaciones, Rotaciones y Escalados, y luego aplicarlas según la secuencia $R \rightarrow S \rightarrow T$, siendo R, S y T las transformaciones que acumulan todas las rotaciones, todos los escalados y todas las traslaciones de manera independiente.

Los objetos deben modelarse centrados en el origen de coordenadas, aunque se solapen unos con otros, para que la acumulación de transformaciones permita una visualización correcta.

Las transformaciones a aplicar serían:

Traslación en X: **“cursor izquierda”** (positiva) y **“derecha”** (negativa)

Traslación en Z: **“cursor arriba”** (positiva) y **“abajo”** (negativa)

Traslación en Y: **“u”** (positiva) y **“U”** (negativa)

Rotación en X: **“x”** (positiva) y **“X”** (negativa)

Rotación en Y: **“y”** (positiva) e **“Y”** (negativa)

Rotación en Z: **“z”** (positiva) y **“Z”** (negativa)

Escalado homogéneo en XYZ: **“s”** (aumento) y **“S”** (disminución)

A cada pulsación de tecla debe visualizarse de modo inmediato la transformación realizada al objeto de la escena.

Para visualizar lo anterior se ha debido utilizar una cámara en una posición concreta. Se debe además permitir el movimiento interactivo de la cámara mediante el teclado. En concreto se deben realizar los movimientos de órbita respecto al origen de coordenadas, el de cabeceo de la cámara y el de rotación de la misma sobre el eje Y respecto a la cámara. Para ello, se deben implementar estas funcionalidades en la clase Cámara, incluyendo las variables y funciones adecuadas para estos tipos de movimiento.

El modo de funcionamiento será el siguiente: tras activar el modo de movimiento de la cámara pulsando **“c/C”** se usarán los cursores izquierdo, derecho, arriba y abajo para los movimientos de órbita y cabeceo y la tecla **“y/Y”** para la rotación sobre el eje Y de la cámara.

La cámara permitirá también el recorte de la escena, incluyendo dicha funcionalidad en la clase cámara, de modo que permita el movimiento de los planos delanteros y traseros del volumen de visión hacia delante y detrás mediante las teclas **“f/F”** para el plano front y **“b/B”** para el plano back. La operación de zoom se realizará mediante las teclas **“+/-”**.

Finalmente se implementarán varios viewport que permitan visualizar la escena desde 2 vistas, una principal que ocupe toda la ventana y otra en un recuadro superpuesto simulando la vista desde arriba de la escena. También debe poderse intercambiar entre proyección paralelo y perspectiva mediante la tecla **“p/P”** en la vista principal, todo ello mediante la modificación adecuada de la clase Cámara.

Tenga en cuenta que los ejemplos explicativos vienen ya resueltos en la práctica y no es necesario implementarlos, su código servirá de base para la entrega de la práctica correspondiente.

Resumen de la práctica:

- Generar 3 objetos distintos en base a primitivas básicas.
- Seleccionar por teclado un objeto y aplicarle una secuencia de transformaciones de manera interactiva mediante teclado.
- Situar una cámara en la escena y moverla mediante el teclado con los distintos tipos de movimientos de cámara, modificando sus diferentes parámetros.
- Realizar operaciones de recorte y de zoom.

- Implementar varios viewport simultáneos.

Entrega de la práctica:

- **Código** en C++ **autodocumentado**, usando las clases proporcionadas.
- **Vídeo** explicativo con la aplicación en funcionamiento.
- Explicación de **detalles fundamentales** de la implementación realizada.

Información importante sobre el envío de la práctica:

- Sólo debe subir los ficheros **uno de los dos miembros** de la pareja de prácticas.
- En el **nombre del fichero** se debe indicar los dos miembros de la pareja, se recomienda un fichero .zip
- Se deben incluir **todos los ficheros con código fuente** .cpp y .h proporcionados aunque no se hayan modificado, de manera que con todos los ficheros pueda crearse un proyecto completo y ejecutarse sin la necesidad de incorporar ficheros adicionales.
- Se deben incluir el resto de ficheros de configuración del proyecto (.cmake, .py, .yaml, .txt), pero ningún otro fichero como .o ó ejecutable .exe.
- Es recomendable incluir en el fichero con la explicación de los detalles fundamentales de funcionamiento, el nombre de los dos miembros del grupo.
- Es preferible incluir en el fichero con la explicación de los detalles fundamentales de funcionamiento un **enlace al vídeo** que sea descargable de un repositorio como puede ser Google Drive. Se debe comprobar que se tienen los **permisos necesarios** para que cualquier docente pueda descargarlo.
- Las prácticas deben ser entregadas en la fecha determinada a principio de curso para poder ser evaluadas y permitir presentarse al examen de la práctica correspondiente.

Ejemplo explicativo para Transformaciones Geométricas (pr1a)

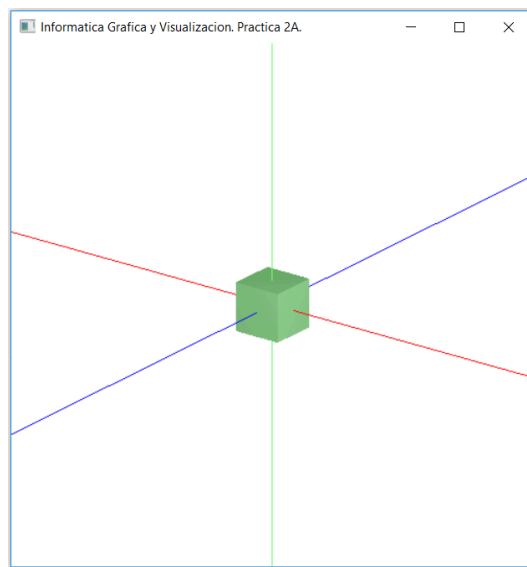
Objetivos:

- Practicar con el concepto de matrices de transformación.
- Aplicar transformaciones: traslación, escalado y rotación.
- Utilizar la composición de matrices.
- Combinar varias transformaciones usando la pila de matrices (obligatorio).

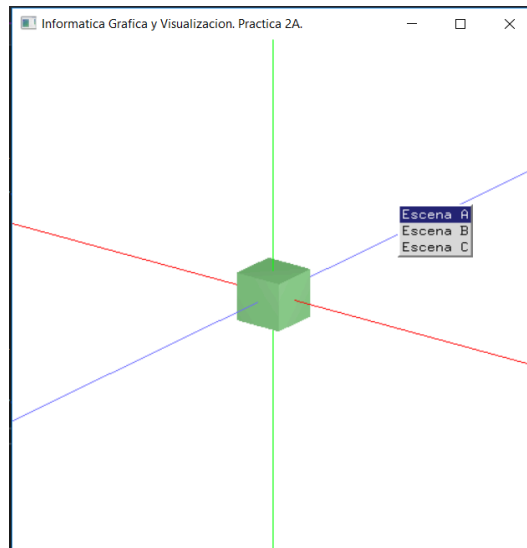
Se parte del código:

- **pr1a.cpp**: función main() del programa
- **igvInterfaz.h** e **igvInterfaz.cpp**: especificación e implementación de la clase **igvInterfaz**, que contiene la funcionalidad básica para crear una ventana de visualización, su configuración y la gestión de los eventos del sistema.
- **igvEscena3D.h** e **igvEscena3D.cpp**: especificación e implementación de la clase **igvEscena3D**, que contiene la funcionalidad básica para visualizar una escena.

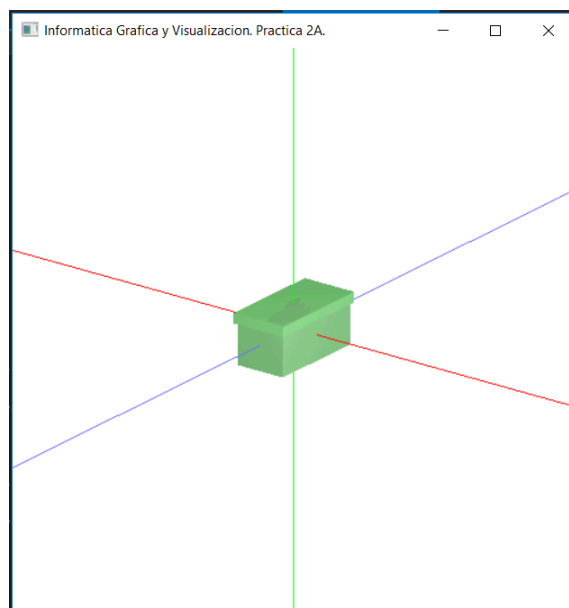
Si ejecutamos el programa sin realizar ninguna modificación se abre una ventana mostrando lo siguiente:



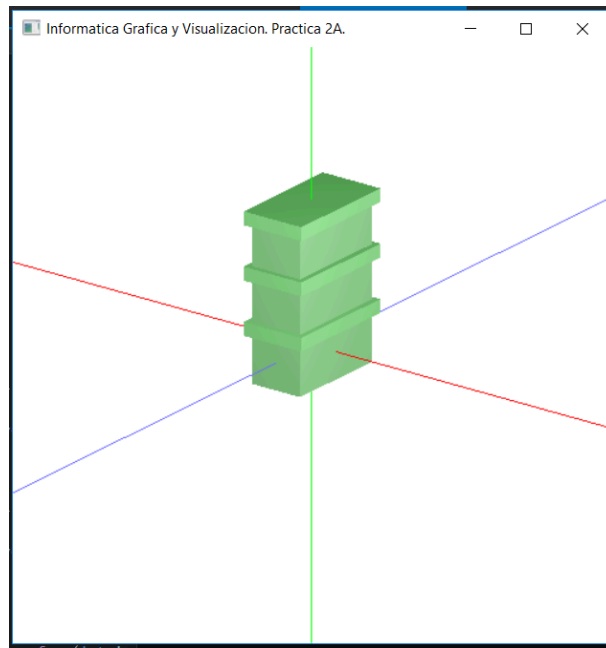
El programa incluye un menú para poder cambiar las escenas (hay una creada para cada ejercicio). Para que aparezca este menú, hay que pulsar el botón derecho del ratón sobre la ventana.



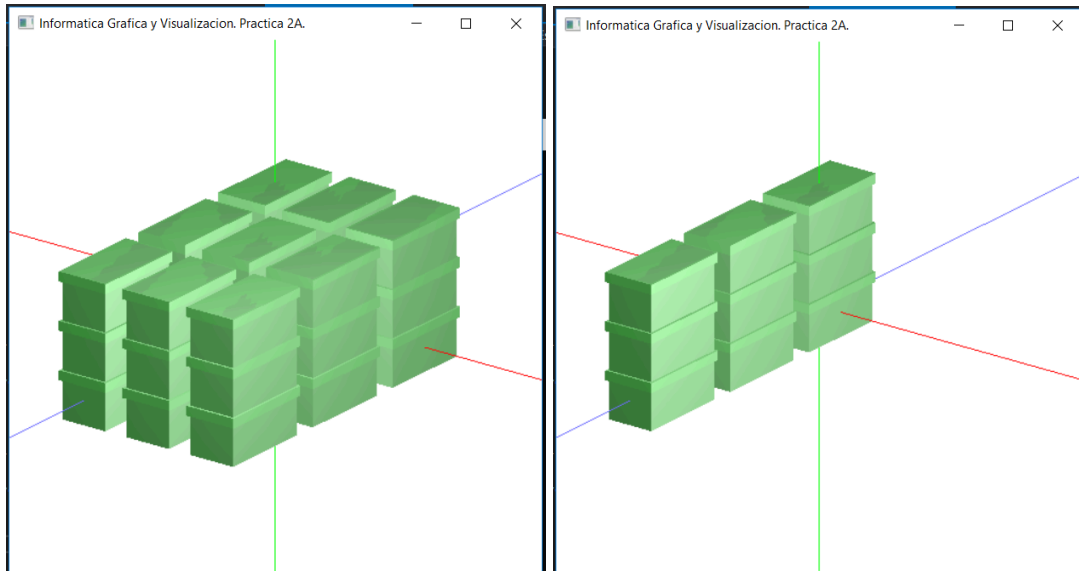
- A. Añadir el código necesario en el método `igvEscena3D::renderEscenaA()` para que visualice la siguiente figura (el uso de la pila de matrices es obligatorio):



- B. Añadir el código necesario en el método `igvEscena3D::renderEscenaB()` para construir una pila con instancias del modelo del apartado A. El número de instancias debe ser parametrizable. En caso de no realizar el apartado C, debe implementarse la pulsación de la tecla "y"/"Y" como viene descrito en dicho apartado.



- C. Modificar el código del método `igvEscena3D::renderEscenaC()` para replicar la pila de instancias a lo largo del eje X y Z interactivamente. Permite aumentar/disminuir el número de instancias en cada pila a través de la pulsación de las correspondientes teclas “x”/”X” para aumentar/disminuir el número de pilas de cajas en el eje X, “z”/”Z” en el eje Z, y “y”/”Y” para aumentar/disminuir el número de cajas en la pila.



Ejemplo explicativo para la Cámara de Visión, Proyecciones y Viewport (pr1b)

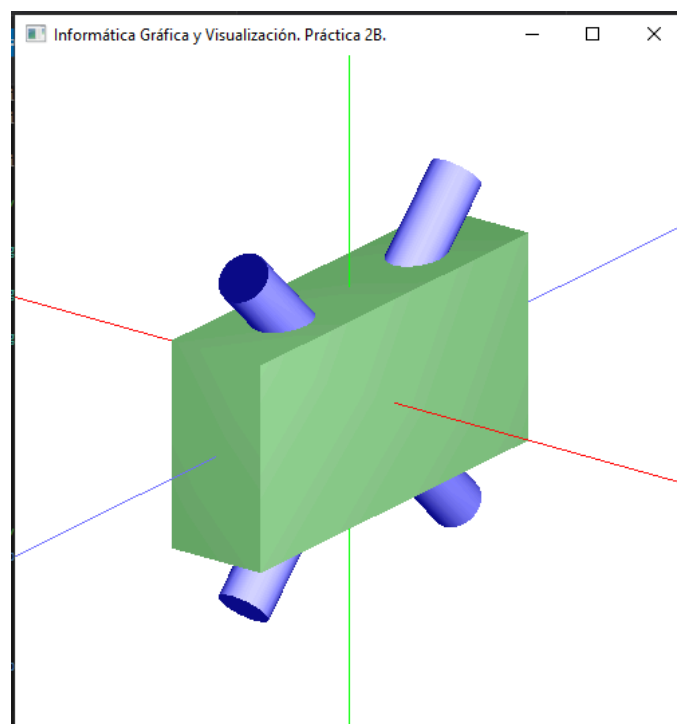
Objetivos:

- Entender el comportamiento de la cámara. Definir y utilizar adecuadamente los parámetros para el posicionamiento y orientación de la cámara de visión.
- Afianzar los conceptos relativos a las proyecciones paralela y en perspectiva.
- Utilización de la transformación de viewport.

Se parte del código:

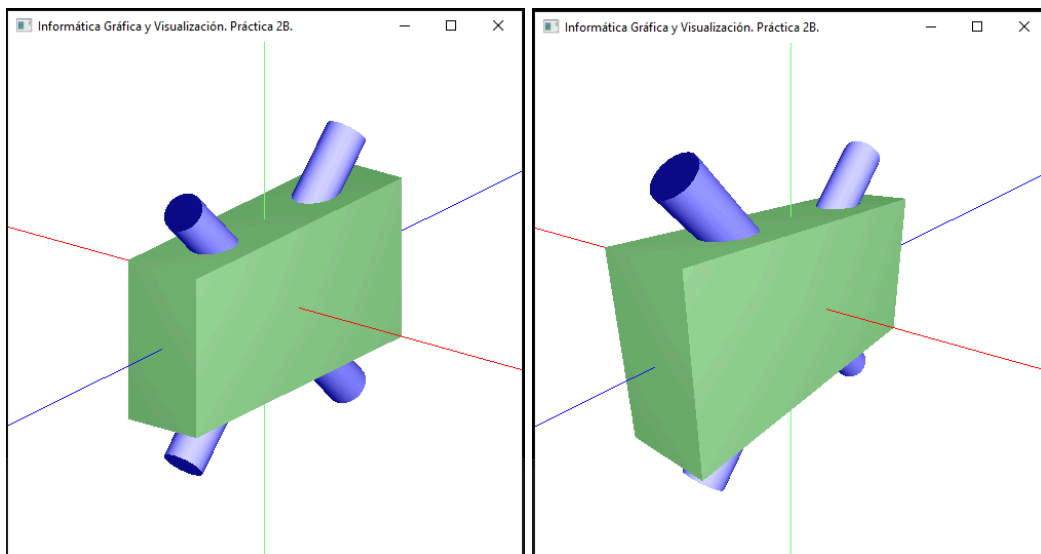
- **pr1b.cpp**: función `main()` del programa
- **igvInterfaz.h** e **igvInterfaz.cpp**: especificación e implementación de la clase *igvInterfaz*, que contiene la funcionalidad básica para crear una ventana de visualización, su configuración y la gestión de los eventos del sistema.
- **igvEscena3D.h** e **igvEscena3D.cpp**: especificación e implementación de la clase *igvEscena3D*, que contiene la funcionalidad básica para visualizar una escena.
- **igvPunto3D.h** e **igvPunto3D.cpp**: especificación e implementación de la clase *igvPunto3D*, que contiene la funcionalidad para declarar y utilizar objetos de tipo punto y vector.
- **igvCamara.h** e **igvCamara.cpp**: especificación e implementación de la clase *igvCamara*, que contiene la funcionalidad básica para crear y manipular cámaras de visión en la aplicación.

Si ejecutamos el programa sin realizar ninguna modificación se abre una ventana mostrando lo siguiente:



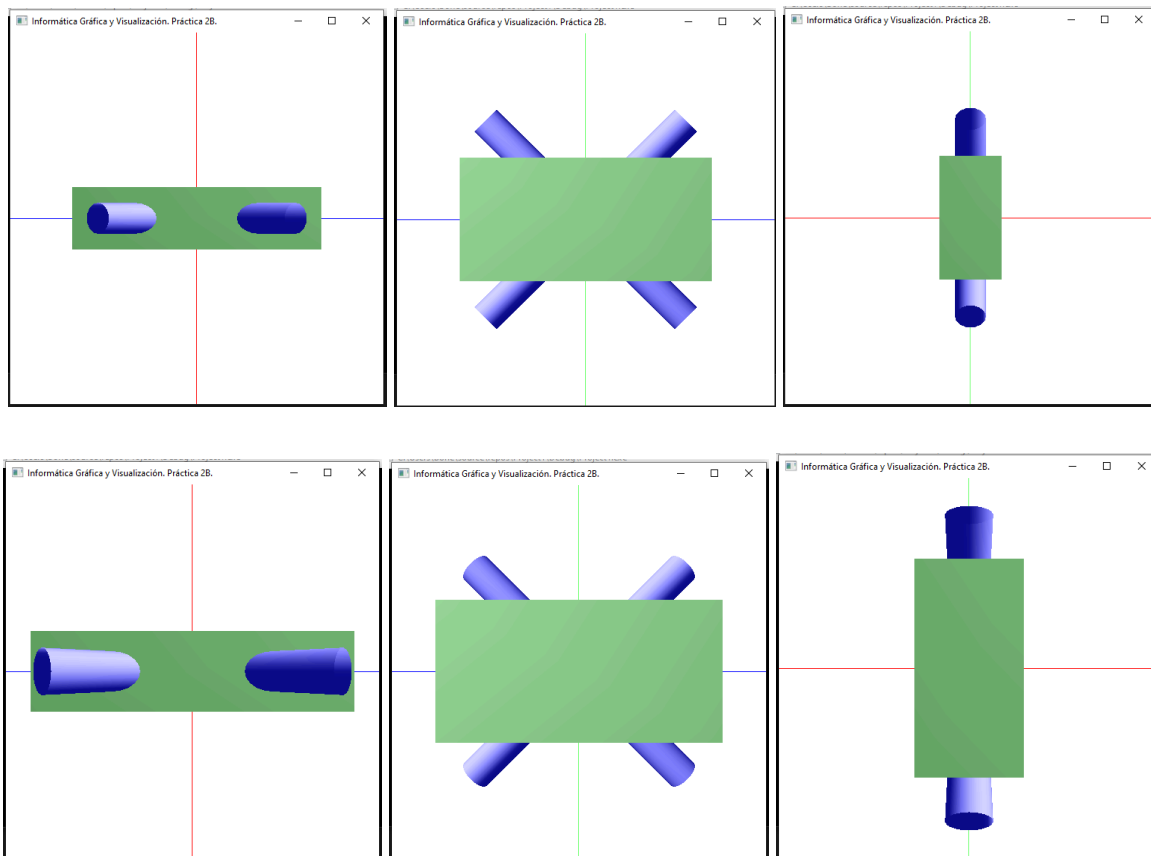
- La aplicación visualiza un objeto centrado en el origen de coordenadas formado por un paralelepípedo de lados 1, 2 y 4, y dos cilindros de radio 0.25 y longitud 4.
- Utiliza una proyección paralela con los parámetros de la cámara:
 - Punto de visión: (3, 2, 4)
 - Punto de referencia: (0, 0, 0)
 - Vector hacia arriba: (0, 1, 0)
- La tecla 'e' activa/desactiva la visualización de los ejes de coordenadas.
- La tecla 'escape' termina la ejecución del programa.

A. Realizar las modificaciones necesarias en el código del método *igvInterfaz::keyboardFunc* para que al pulsar la tecla 'P/p' intercambie el tipo de proyección de tipo paralela a tipo perspectiva (ángulo y aspecto) y viceversa. La posición de la cámara tiene que mantenerse común en ambas proyecciones:



- B. Además de la vista panorámica original, se deben añadir nuevas vistas del modelo: planta (punto de visión en el eje Y), perfil (punto de visión en el eje X) y alzado (punto de visión en el eje Z). Cambia la inicialización del atributo *igvInterfaz::camara* para representar estas vistas adicionales.
- Modifica el método *igvInterfaz::keyboardFunc* para poder cambiar interactivamente, pulsando la tecla 'V/v', las vistas del modelo.
 - Si se está en un tipo de proyección y se pulsa la tecla 'P/p' (apartado A), se debe obtener la misma vista, pero cambiando el tipo de proyección.

Las diferentes vistas para cada tipo de proyección deben aparecer como se muestra en las siguientes figuras (la vista en panorámica inicial se corresponderá con la figura del apartado A):

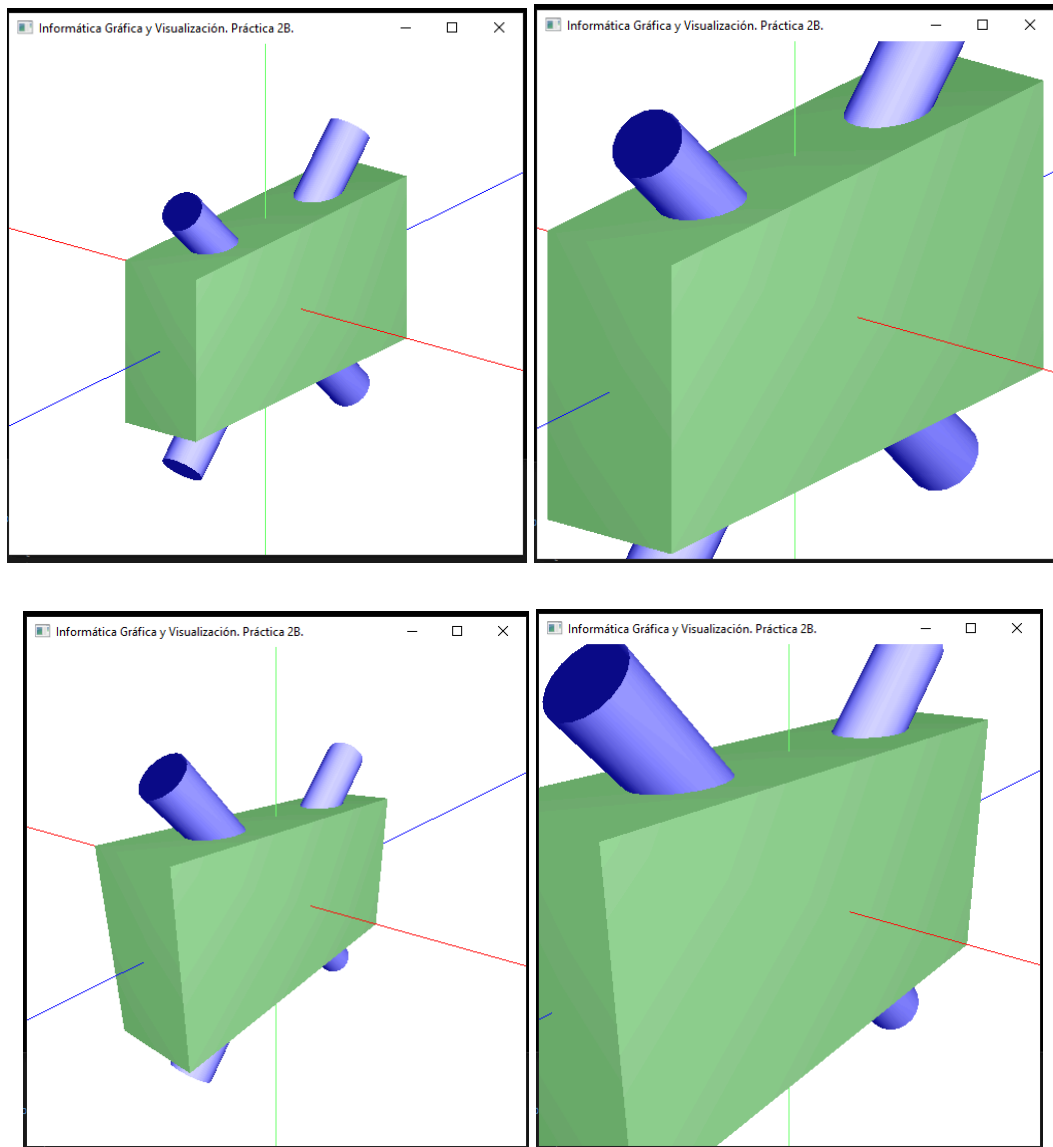


C. Implementar el método `void igvCamara::zoom(double factor)` para que realice un zoom de la cámara.

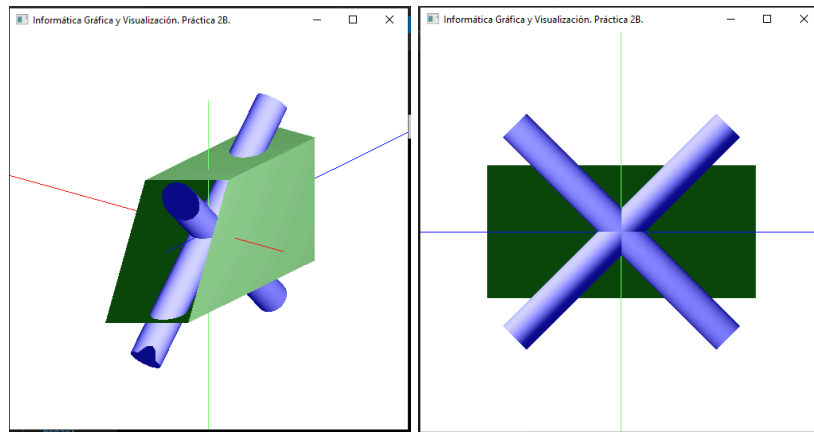
- El zoom se debe implementar de tal manera que los parámetros de la cámara se modifiquen para que el tamaño de la imagen se acerque o aleje en función del valor del parámetro *factor* (porcentaje de zoom).
- La posición de la cámara no debe modificarse al aplicar el zoom.
- El zoom debe funcionar tanto en la proyección paralela como en la proyección perspectiva.
- Las coordenadas del objeto no deben modificarse al hacer el zoom.

Añade el código necesario a los métodos `igvInterfaz::keyboardFunc` y `igvCamara::zoom(double factor)` para poder cambiar interactivamente el porcentaje de zoom utilizando las teclas '+' (acercarse) y '-' (alejarse). Cada pulsación de estas teclas implicará un factor del 5% del zoom (incrementar o disminuir).

Como ejemplo, si pulsamos 10 veces la tecla '+' se debe obtener la visualización que se muestra en la siguiente figura para las proyecciones paralela y perspectiva. Pulsando después 10 veces la tecla '-' se debe volver a la situación de partida:



- D. Añadir el código necesario en el método `igvInterfaz::keyboardFunc` para poder cambiar interactivamente la distancia del plano cercano del volumen de visión definido por la cámara (tanto en proyección paralela como en perspectiva). Cada pulsación de la tecla 'n' incrementará el valor de la distancia del plano cercano en 0.2 unidades y al pulsar la tecla 'N' se disminuirá el valor en esa misma cantidad. Después de pulsar varias veces la tecla 'n' el plano cercano debe atravesar el modelo y mostrarse visualizaciones como las siguientes (proyección paralela panorámica y proyección perspectiva en planta):



E. Añadir el código necesario en el método *igvInterfaz::keyboardFunc* y modifica el método *igvInterfaz::displayFunc* y la clase *igvInterfaz* para dividir la ventana en cuatro vistas, pulsando la tecla '4', y teniendo en cuenta las siguientes restricciones:

- Los parámetros de la cámara no deben modificarse.
- Si la tecla '4' se vuelve a pulsar se restaurará el formato original.
- Toda la funcionalidad de esta práctica debe funcionar con las 4 vistas.
- Esta configuración se tiene que adaptar al tamaño de la pantalla.

